LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 – INHERITANCE



Ardian Nafis Samudra

103132400012

PS1SD-05-reg-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

- **Inheritance (Pewarisan):** Proses di mana suatu kelas diturunkan dari kelas lainnya sehingga kelas turunan (subclass) mendapatkan ciri, atribut, atau perilaku (behaviour) dari kelas dasar (superclass), yang bertujuan untuk menghindari duplikasi kode.
- **Keyword super:** Kata kunci yang digunakan untuk mereferensikan superclass dari suatu kelas, baik untuk memanggil constructor kelas induk secara spesifik maupun untuk mengakses atribut dan method yang ada di dalam superclass.
- **Method Overloading:** Kemampuan untuk membuat dua atau lebih method dengan nama yang sama di dalam satu kelas yang sama, tetapi memiliki perbedaan pada jumlah, urutan, atau tipe data argumen parameternya.
- **Method Overriding:** Kemampuan subclass untuk mendefinisikan ulang sebuah method yang sama persis dengan yang dimiliki oleh superclass-nya, sehingga saat dipanggil melalui objek subclass, method milik subclass-lah yang akan dieksekusi.
- **Polymorphism (Polimorfisme):** Kemampuan dari suatu objek untuk mengambil beberapa bentuk yang berbeda, yang memungkinkan penggunaan referensi kelas induk yang sama untuk memanggil aksi atau tindakan yang secara proses di dalam kelas turunannya berbeda.

B. Guided

1. Guided1 (A.java, B.java, & main.java)

```
package Guided1;

public class A {
    int x;
    int y;

    void tampilXY(){
        System.out.println("Nilai x = "+x+", y = "+y);
    }
            }
```

```
package Guided1;

public class B extends A{
    int z;
    int hasil;

    void jumlahXYZ(){
        hasil = (x+y+z);

        System.out.println("Jumlah (x+y+z))= " + hasil);
```

```
    }

}
```

```
package Guided1;

public class Demo {

    public static void main(String[] args){

        A superclass = new A();
        B subclass = new B();

        superclass.x = 3;
        superclass.y = 4;

        System.out.println("Superclass: ");
        superclass.tampilXY();

        subclass.x = 1;
        subclass.y = 2;
        subclass.z = 5;

        System.out.println("Subclass: ");
        subclass.jumlahXYZ();
    }


}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Praktikummgg06 ---
Superclass:
Nilai x = 3, y = 4
Subclass:
Jumlah (x+y+z))= 8
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.262 s
Finished at: 2026-10-24T08:43:09+07:00
------------------------------------------------------------------------
```

2. Guided2 (Employee.java, Manager.java, & Demo.java)

```
package Guided2;

public class Employee {
    private String nama;
    String departemen;

    public Employee(String nama){
        this.nama = nama;
    }

    public void tampilNama(){
        System.out.println("nama :"+ nama);
    }

                }
```

```
package Guided2;

public class Manager extends Employee{
    private String alamat;

    public Manager(String nama, String alamat){
        super(nama);
        this.alamat = alamat;
    }

    public void tampilInformasi(){
        super.tampilNama();
        System.out.println("alamat: " + alamat);
        System.out.println("departemen: " + super.departemen);
    }


}
```

```
package Guided2;

public class Demo {
    public static void main(String[] args){
        Manager jarwo = new Manager("Jarwo","sukabirus");
        jarwo.departemen = "SDM";
        jarwo.tampilInformasi();
    }

}
```

Output :

```
nama :Jarwo
alamat: sukabirus
departemen: SDM
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.318 s
Finished at: 2026-10-24T08:45:27+07:00
------------------------------------------------------------------------
```

3. Guided3 (Lagu.java & main.java)

```
package Guided3;

public class lagu {
    String pencipta;
    String judul;

    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1, String param2) {
        judul = param1;
        pencipta = param2;
    }

    void tampilkan() {
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
    }
            }
```

```
package Guided3;

public class main {
    public static void main(String[] args) {
        lagu d, e;
        d = new lagu();
        e = new lagu();

        d.IsiParam("Lagu 1");
```

```
        e.IsiParam("kepastian yang kutunggu", "GiGi");

        d.tampilkan();
        e.tampilkan();
    }
}
```

Output :

```
Judul : Lagu 1, pencipta : Tidak dikenal
Judul : kepastian yang kutunggu, pencipta : GiGi
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.617 s
Finished at: 2026-10-24T08:47:41+07:00
------------------------------------------------------------------------
```

4. Guided4 (A.java, B.java, & main.java)

```
package Guided4;

public class A {
    public void tampilkanKeLayar() {
        System.out.println("Method milik class A dipanggil...");
    }
            }
```

```
package Guided4;

public class B extends A{
    public void tampilkanKeLayar() {
        super.tampilkanKeLayar();
        System.out.println("Method milik class B dipanggil...");
    }
}
```

```
package Guided4;

public class main {
    public static void main(String[] args) {
        B subOb = new B();
        subOb.tampilkanKeLayar();
    }
}
```

Output :

```
Method milik class A dipanggil...
Method milik class B dipanggil...
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.614 s
Finished at: 2026-10-24T08:51:01+07:00
------------------------------------------------------------------------
```

C. Unguided

- Sebuah tim machine learning ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih.
- Ada tiga jenis model yang digunakan: model regresi, model klasifikasi, dan model klasifikasi berbasis deep learning yang merupakan pengembangan dari model klasifikasi biasa.
- Buatlah program Java dengan ketentuan: Penamaan kelas, atribut, dan method ditentukan sendiri oleh Anda dan semua atribut bersifat private.

Ketentuan Program:

1. Kelas induk (model umum):
    - Menyimpan nama model (teks) dan jumlah data latih (bilangan bulat).
    - Memiliki constructor yang menerima kedua nilai tersebut, serta cara bagi kelas lain untuk membaca jumlah data latih.
    - Memiliki method untuk menampilkan informasi.
    - Memiliki dua method untuk menghitung biaya komputasi dengan nama yang sama (overloading):
        - Tanpa parameter: biaya = jumlah data latih * 2.
        - Dengan satu parameter bilangan bulat / jumlah epoch: biaya = jumlah data latih * epoch.
2. Kelas turunan pertama (model regresi):
    - Mewarisi kelas induk dan menambahkan atribut jumlah fitur.
    - Constructor menerima nama, jumlah data latih, dan jumlah fitur; gunakan constructor kelas induk.
    - Override method: tampilkan informasi dari kelas induk.
    - Override method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) + (jumlah fitur * 10).
    - Menambahkan satu method baru dengan nama yang sama dengan method penghitung biaya, tetapi menerima dua parameter bilangan bulat (jumlah epoch dan ukuran batch): biaya = (jumlah data latih * epoch) / ukuran batch.
    - Sehingga objek kelas ini memiliki tiga versi method penghitung biaya.

3. Kelas turunan kedua (model klasifikasi):
    - Mewarisi kelas induk dan menambahkan atribut jumlah kelas.
    - Constructor menerima nama, jumlah data latih, dan jumlah kelas; gunakan constructor kelas induk.
    - Override method: tampilkan informasi dari kelas induk.
    - Override method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) * jumlah kelas.
    - Versi penghitung biaya dengan parameter epoch tidak di-override.
4. Kelas turunan ketiga (model klasifikasi deep learning):
    - Mewarisi kelas turunan kedua (bukan kelas induk langsung) dan menambahkan atribut jumlah layer.
    - Constructor menerima nama, jumlah data latih, jumlah kelas, dan jumlah layer; gunakan constructor kelas di atasnya.
    - Override method: tampilkan informasi dari kelas di atasnya.
    - Override method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas di atasnya) + (jumlah layer * 200).
5. Class Main:
    - Membuat tiga objek berikut:
        - Regresi: Nama "Regresi Linier", Data latih 500, Atribut tambahan 4 fitur.
        - Klasifikasi: Nama "Decision Tree", Data latih 800, Atribut tambahan 3 kelas.
        - Klasifikasi deep learning: Nama "Neural Network", Data latih 1000, Atribut tambahan 5 kelas, 3 layer.
    - Simpan ketiga objek tersebut dalam satu array bertipe kelas induk.
    - Gunakan perulangan untuk setiap elemen array: tampilkan informasi model, tampilkan biaya komputasi, lalu jumlahkan seluruh biaya.
    - Setelah perulangan, tampilkan total biaya.
    - Terakhir, tampilkan juga:
        - Biaya 10 epoch untuk model "Neural Network" (ambil dari array).
        - Biaya 10 epoch untuk model "Regresi Linier".
        - Biaya 10 epoch dengan ukuran batch 50 untuk model "Regresi Linier"

Jawaban :

1. Unguided (ModelML.java, ModelRegresi.java, ModelKlasifikasi.java, ModelDeepLearning.java, & Main.java)

```
package Unguided;

public class ModelML {
    private String namaModel;
    private int jumlahDataLatih;

    // constructor
    public ModelML(String namaModel, int jumlahDataLatih) {
        this.namaModel = namaModel;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public String getNamaModel() {
        return namaModel;
    }

    public int getJumlahDataLatih() {
        return jumlahDataLatih;
    }

    // method menampilkan informasi
    public void tampilkanInfo() {
        System.out.println("Model: " + namaModel + " | Data latih: " + jumlahDataLatih);
    }

    // overloading: hitung biaya tanpa parameter
    public int hitungBiaya() {
        return jumlahDataLatih * 2;
    }

    // overloading: hitung biaya dengan parameter epoch
    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

```
package Unguided;

public class ModelRegresi extends ModelML {
    private int jumlahFitur;
```

```
    public ModelRegresi(String namaModel, int jumlahDataLatih, int jumlahFitur) {
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    // 2 parameter: epoch dan ukuran batch
    public int hitungBiaya(int epoch, int ukuranBatch) {
        return (getJumlahDataLatih() * epoch) / ukuranBatch;
    }
}
```

```
package Unguided;

public class ModelKlasifikasi extends ModelML {
    private int jumlahKelas;

    public ModelKlasifikasi(String namaModel, int jumlahDataLatih, int jumlahKelas) {
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }
}
```

```
package Unguided;

public class ModelDeepLearning extends ModelKlasifikasi {
    private int jumlahLayer;
```

```
    public ModelDeepLearning(String namaModel, int jumlahDataLatih, int jumlahKelas, int
jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

```
package Unguided;

public class Main {
    public static void main(String[] args) {
        ModelML regresi = new ModelRegresi("Regresi Linier", 500, 4);
        ModelML klasifikasi = new ModelKlasifikasi("Decision Tree", 800, 3);
        ModelML deepLearning = new ModelDeepLearning("Neural Network", 1000, 5, 3);

        // menyimpan objek dalam satu array bertipe kelas induk (polimorfisme)
        ModelML[] daftarModel = {regresi, klasifikasi, deepLearning};

        int totalBiaya = 0;

        // perulangan untuk setiap elemen array
        for (ModelML model : daftarModel) {
            model.tampilkanInfo();
            int biaya = model.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");
            totalBiaya += biaya;
        }

        // tampilkan total biaya dan uji coba method khusus
        System.out.println("Total biaya: " + totalBiaya);
        System.out.println("Biaya 10 epoch (Neural Network): " +
deepLearning.hitungBiaya(10));
        System.out.println("Biaya 10 epoch (Regresi Linier): " + regresi.hitungBiaya(10));
```

```
        // downcasting untuk mengakses method dengan 2 parameter pada Regresi
        ModelRegresi regresiSpesifik = (ModelRegresi) regresi;
        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): " +
regresiSpesifik.hitungBiaya(10, 50));
    }
}
```

Output :

```
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  4.487 s
Finished at: 2026-10-24T08:57:52+07:00
------------------------------------------------------------------------
```

Penjelasan :

Output program Unguided Modul 6 menunjukkan keberhasilan implementasi konsep inheritance, overriding, overloading, dan polymorphism dalam pemrograman berorientasi objek, di mana informasi setiap model beserta atribut spesifiknya berhasil ditampilkan secara dinamis. Total keseluruhan biaya komputasi sebesar 16440 diakumulasikan secara otomatis melalui perulangan array kelas induk, sementara pengujian method khusus dengan variasi parameter epoch dan ukuran batch menghasilkan nilai yang akurat sesuai dengan ketentuan soal, menandakan program telah berjalan dan dikompilasi dengan sukses (BUILD SUCCESS).

D. Kesimpulan

- **Konsep Pewarisan (Inheritance) dan Keyword super:** Pemahaman dan penerapan pewarisan dalam Pemrograman Berorientasi Objek berhasil diimplementasikan, di mana kelas turunan (subclass) dapat mewarisi atribut dan behaviour dari kelas induk (superclass), serta memanfaatkan keyword super untuk mengakses konstruktor dan member dari kelas di atasnya.
- **Penerapan Method Overloading dan Overriding:** Konsep overloading berhasil diterapkan melalui pembuatan beberapa method berpenamaan sama dengan variasi parameter berbeda dalam satu kelas, sementara overriding diwujudkan dengan menulis ulang method pada subclass guna menyesuaikan rumus perhitungan dan karakteristik spesifik model.
- **Penerapan Polimorfisme (Polymorphism):** Penggunaan array bertipe kelas induk (ModelML) terbukti mampu menampung berbagai objek dari kelas turunan secara fleksibel, memungkinkan pemanggilan method secara dinamis serta pengakumulasian total biaya komputasi secara otomatis.
- **Studi Kasus Estimasi Biaya Komputasi (Unguided):** Tugas unguided berhasil diselesaikan dengan membangun hierarki kelas berjenjang (model umum, regresi, klasifikasi, hingga deep learning) untuk menghitung biaya komputasi berdasarkan atribut data latih, jumlah fitur, kelas, layer, epoch, dan ukuran batch secara akurat.
- **Modularitas dan Pengorganisasian Proyek:** Pengelompokan kelas ke dalam package terpisah (Guided1, Guided2, Guided3, Guided4, dan Unguided) menjadikan struktur proyek Java pada NetBeans lebih terorganisir, rapi, modular, dan mudah dikelola.